A. Supreeth Reddy

2303A51392

Batch 06

AIAC Lab 6.3

Task Description #1: Classes (Student Class)
Scenario
You are developing a simple student information management module.
Task
• Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
• The class should include attributes such as name, roll number, and branch.
• Add a method display_details() to print student information.
• Execute the code and verify the output.
• Analyze the code generated by the AI tool for correctness and clarity.
Expected Output #1
• A Python class with a constructor (__init__) and a display_details() method.
• Sample object creation and output displayed on the console.
• Brief analysis of AI-generated code

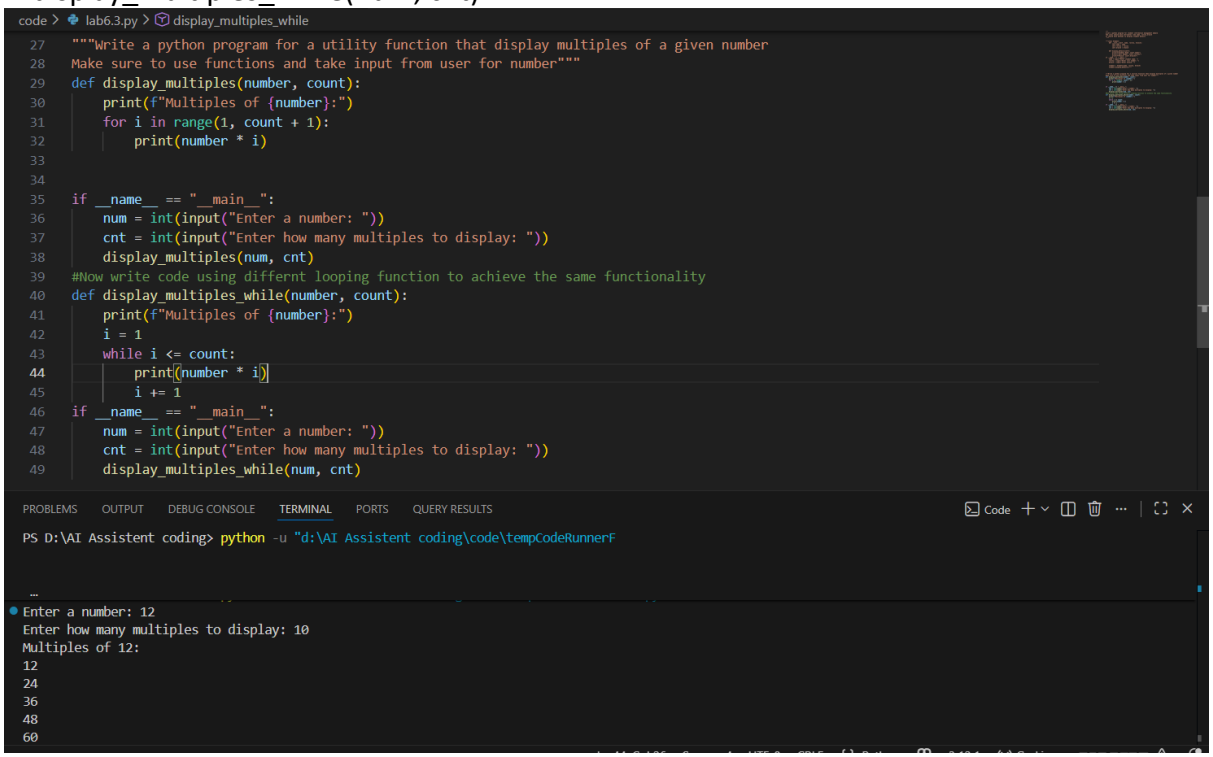
```python
#write a python program for student information managament module

#The class should have attributes like name, rollno,branch

#It should have methods to display student details

class Student:

    def __init__(self, name, rollno, branch):

        self.name = name

        self.rollno = rollno

        self.branch = branch


    def display_details(self):

        print(f"Name: {self.name}")

        print(f"Roll No: {self.rollno}")

        print(f"Branch: {self.branch}")

# Example usage

#user input for student details

name = input("Enter student name: ")

rollno = input("Enter student roll number: ")

branch = input("Enter student branch: ")

student = Student(name, rollno, branch)

print("\nStudent Details:")

student.display_details()
```

```
Enter student name: Supreeth
Enter student roll number: 1392
Enter student branch: cse

Student Details:
Name: Supreeth
Roll No: 1392
Branch: cse
```

Task Description #2: Loops (Multiples of a Number)
Scenario
You are writing a utility function to display multiples of a given number.
Task
• Prompt the AI tool to generate a function that prints the first 10 multiples of a given number
using a loop.
• Analyze the generated loop logic.
• Ask the AI to generate the same functionality using another controlled looping structure (e.g.,
while instead of for).
Expected Output #2
• Correct loop-based Python implementation.
• Output showing the first 10 multiples of a number.
• Comparison and analysis of different looping approaches.
Prompt: Write a python program for a utility function that display multiples of a given number
Make sure to use functions and take input from user for number
Code:

```
"""Write a python program for a utility function that display multiples of a given number
Make sure to use functions and take input from user for number"""
def display_multiples(number, count):
    print(f"Multiples of {number}:")
    for i in range(1, count + 1):
        print(number * i)

if __name__ == "__main__":
    num = int(input("Enter a number: "))
    cnt = int(input("Enter how many multiples to display: "))
    display_multiples(num, cnt)
#Now write code using differnt looping function to achieve the same functionality
def display_multiples_while(number, count):
    print(f"Multiples of {number}:")
    i = 1
    while i <= count:
        print(number * i)
        i += 1
if __name__ == "__main__":
```

```
num = int(input("Enter a number: "))
cnt = int(input("Enter how many multiples to display: "))
display_multiples_while(num, cnt)
```



Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

• Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).

• Analyze the generated conditions and logic.

• Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Expected Output #3

• A Python function that classifies age into appropriate groups.

• Clear and correct conditional logic.

• Explanation of how the conditions work

Prompt: """Write a python function to classify age groups

It should use nested if else statements to classify age groups as follows:

- Child: 0-12
- Teen: 13-19
- Adult: 20-59
- Senior: 60 and above

The function should take age as input and return the age group as output"""

"""Now write function to generate same output using alternative approach without using nested if else statements"""

Code:

"""Write a python function to classify age groups

It should use nested if else statements to classify age groups as follows:
- Child: 0-12
- Teen: 13-19
- Adult: 20-59
- Senior: 60 and above

The function should take age as input and return the age group as output"""

```
def classify_age_group(age):
    if age >= 0:
        if age <= 12:
            return "Child"
        else:
            if age <= 19:
                return "Teen"
            else:
                if age <= 59:
                    return "Adult"
                else:
                    return "Senior"
    else:
        return "Invalid age"
```

"""Now write function to generate same output using alternative approach without using nested if else statements"""

```
def classify_age_group_alternative(age):
    if age < 0:
        return "Invalid age"
    elif age <= 12:
        return "Child"
    elif age <= 19:
        return "Teen"
    elif age <= 59:
        return "Adult"
    else:
        return "Senior"
if __name__ == "__main__":
    age = int(input("Enter age: "))
    group = classify_age_group(age)
    print(f"Age group (nested if-else): {group}")

    group_alt = classify_age_group_alternative(age)
    print(f"Age group (alternative approach): {group_alt}")
```

```
code > lab6.3.py > ...
- Senior: 60 and above
The function should take age as input and return the age group as output"""
def classify_age_group(age):
    if age >= 0:
        if age <= 12:
            return "Child"
        else:
            if age <= 19:
                return "Teen"
            else:
                if age <= 59:
                    return "Adult"
                else:
                    return "Senior"
    else:
        return "Invalid age"
"""Now write function to generate same output using alternative approach without using nested if else statements"""
def classify_age_group_alternative(age):
    if age < 0:
        return "Invalid age"
    elif age <= 12:
        return "Child"
    elif age <= 19:
        return "Teen"
    elif age <= 59:
        return "Adult"
    else:
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY RESULTS                Code +

PS D:\AI Assistent coding> python -u "d:\AI Assistent coding\code\lab6.3.py"
Enter age: 13
Age group (nested if-else): Teen
Age group (alternative approach): Teen
PS D:\AI Assistent coding>
```

Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a sum_to_n() function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

Prompt: Write a python program to calculate sum of n natural numbers using function sum_to_n() using a for loop

Code:

```
def sum_to_n(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
if __name__ == "__main__":
    n = int(input("Enter a natural number: "))
    result = sum_to_n(n)
    print(f"Sum of first {n} natural numbers is: {result}")
```

```python


"""Write a python program to calculate sum of n natural numbers using function sum_to_n() using a for loop
"""
def sum_to_n(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
if __name__ == "__main__":
    n = int(input("Enter a natural number: "))
    result = sum_to_n(n)
    print(f"Sum of first {n} natural numbers is: {result}")
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS Code

```
PS D:\AI Assistent coding> python -u "d:\AI Assistent coding\code\lab6.3.py"
Enter age: 13
Age group (nested if-else): Teen
Age group (alternative approach): Teen
PS D:\AI Assistent coding>
```

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().

• Analyze the AI-generated class structure and logic.

• Add meaningful comments and explain the working of the code.

Expected Output #5

• Complete Python Bank Account class.

• Demonstration of deposit and withdrawal operations with updated balance.

• Well-commented code with a clear explanation.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

Prompt: Wrirte a python program to basic banking applicattion

It should have a bank account class with methods to deposit, withdraw and check balance

Code:

```
class BankAccount:
    def __init__(self, account_holder, initial_balance=0):
        self.account_holder = account_holder
        self.balance = initial_balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: ${amount:.2f}")
        else:
            print("Deposit amount must be positive.")
```

```
    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew: ${amount:.2f}")
        else:
            print("Insufficient balance or invalid withdrawal amount.")

    def check_balance(self):
        print(f"Current balance: ${self.balance:.2f}")
if __name__ == "__main__":
    name = input("Enter account holder name: ")
    account = BankAccount(name)

    while True:
        print("\nOptions:")
        print("1. Deposit")
        print("2. Withdraw")
        print("3. Check Balance")
        print("4. Exit")
        choice = input("Choose an option (1-4): ")

        if choice == '1':
            amount = float(input("Enter amount to deposit: "))
            account.deposit(amount)
        elif choice == '2':
            amount = float(input("Enter amount to withdraw: "))
            account.withdraw(amount)
        elif choice == '3':
            account.check_balance()
        elif choice == '4':
            print("Exiting the banking application.")
            break
        else:
            print("Invalid option. Please try again.")
```

```python
class BankAccount:
    def __init__(self, account_holder, initial_balance=0):
        self.account_holder = account_holder
        self.balance = initial_balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: ${amount:.2f}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew: ${amount:.2f}")
        else:
            print("Insufficient balance or invalid withdrawal amount.")

    def check_balance(self):
        print(f"Current balance: ${self.balance:.2f}")
if __name__ == "__main__":
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS D:\AI Assistent coding> python -u "d:\AI Assistent coding\code\lab6.3.p
...
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose an option (1-4): 1
Enter amount to deposit: 200
Deposited: $200.00
```